

Alinhamento do Projeto Integrador — Alexandria à luz da disciplina de Testes de Software

Projeto: Alexandria — Biblioteca Digital **Instituição:** PUC-SP — MBA em Engenharia de Software **Disciplina:** Testes de Software — Prof. Jehú Lara Ramos **Data:** 08/08/2026 **Base:** Aulas 1 a 4
(D:\projetos\puc\semestre_2\testes de software), com foco no item 15 da Aula 4 — *Aviso: primeira aula de Projeto Integrador no sábado*

Sumário Executivo

Foi pedido, ao final da Aula 4, que cada grupo apresente na sessão de sábado a **estrutura do projeto** — não o escopo, que já considera claro, mas o detalhe das tasks e issues: se são histórias bem formadas, se as personas estão mapeadas, em qual ferramenta o grupo trabalha e como os testes se organizam a partir disso.

Cruzando esse pedido com o estado real do repositório, o diagnóstico é direto:

O Alexandria é tecnicamente forte e estruturalmente frágil. A base da pirâmide de testes está acima da média (245 testes unitários, 85% de cobertura, arquitetura hexagonal disciplinada), mas a estrutura que deveria dar sentido a esses testes — requisitos consistentes, histórias, personas, rastreabilidade — está ausente ou contraditória.

O grupo **tem** um Jira, com 27 issues e 4 sprints, usado de abril a meados de maio de 2026 e descontinuado desde então. E **tem** oito histórias de usuário bem formadas, escritas na concepção do produto — que nunca chegaram à ferramenta.

O achado mais grave da análise decorre disso: **existem três definições de requisito paralelas e divergentes** — as user stories, a lista RF01–RF10 do Jira e outra lista RF01–RF10 na documentação, com identificadores que colidem (RF08 é "Registrar empréstimo" num lugar e "Leitor de PDF" no outro). Nenhuma descreve o produto em produção. Enquanto isso não for resolvido, nenhuma matriz de rastreabilidade é construível.

O padrão de fundo é consistente: **o grupo domina as práticas, mas não as sustentou no tempo.** Sabe escrever história de usuário (as 8 são melhores que o backlog do Jira), sabe estruturar sprint (a Sprint 3 fechou no prazo), sabe testar (245 unitários, 85% de cobertura) — e cada uma dessas coisas foi abandonada num ponto diferente do caminho.

Esta é uma sessão de revisão e alinhamento, não uma apresentação avaliativa. A recomendação é entrar com o diagnóstico assumido, não maquiado — o valor da sessão está justamente em usar os gaps reais como material de discussão.

1. O que o professor pediu (Aula 4, item 15)

#	Pedido explícito	Resposta atual do Alexandria
1	Qual ferramenta de gestão o grupo usa (Jira, Azure	Jira (alexandria-puc.atlassian.net , projeto Alexandria), usado de 02/04 a meados de maio/2026 e descontinuado desde então . A

#	Pedido explícito	Resposta atual do Alexandria
	DevOps...) e se pretende continuar nela	decisão de retomar ou migrar para GitHub Projects está em aberto — ver analise-jira.md
2	As tasks e issues são de fato histórias bem formadas?	No Jira, não — das 27 issues, as 3 tipadas como "História" são tarefas técnicas, sem critérios de aceite. Mas existem 8 histórias bem formadas (US01–US08, formato "Como/Quero/Para", com critérios de aceite e 4 épicos de negócio), escritas na concepção e nunca registradas na ferramenta — ver user-stories.md
3	As personas estão mapeadas?	Não . As 8 histórias usam uma persona única e genérica ("Como usuário"). Há 4 perfis em "Público-Alvo" (docs/documentacao.md:67), mas não convertidos em personas nem em jornadas
4	Como os testes estão organizados a partir disso?	Organizados a partir do código , não dos requisitos. Não existe vínculo requisito → caso de teste → evidência, e 0 de 229 commits citam uma chave do Jira

O objetivo declarado pelo professor é revisar junto com os grupos as histórias e tasks, para que os temas da disciplina possam ser trabalhados com exemplos reais tirados do próprio projeto integrador. Isso significa que os gaps acima **são o insumo da conversa**, não um problema a esconder.

2. Diagnóstico contra os seis pilares das Aulas 1–4

O professor abre a Aula 4 recapitulando seis pilares construídos nas três aulas anteriores. Mapeando cada um contra o estado do Alexandria:

Pilar	Aula	Situação no Alexandria	Status
Shift left — antecipar qualidade ao upstream	1	Nenhum rito de refinamento, três amigos ou DoR documentado. Qualidade concentrada no downstream (código + CI)	
Origem dos defeitos — requisitos e comunicação	1, 2, 3	Três definições de requisito conflitantes — as user-stories.md , o Jira (SCRUM-17) e docs/documentacao.md , com os mesmos IDs RF01–RF10 significando coisas diferentes. Sem critérios de aceite objetivos, sem BDD	
Estrutura de projeto — Initiative → Epic → Feature → Story → Subtask	2, 3	Existe no Jira (4 épicos, 22 issues com parent), mas os épicos são organizados por camada técnica ("Frontend React", "Desenvolvimento Backend") e não por valor de negócio — o que inviabiliza medir cobertura por objetivo	
Equipes e alinhamento — papéis, acordos de trabalho	2, 3	Não documentado. Não há papel de QA explicitado, nem working agreements	
Requisitos, rastreabilidade e cobertura	2, 3	Cobertura técnica excelente (85% JaCoCo); rastreabilidade zero — nenhum teste referencia um RF	

Pilar	Aula	Situação no Alexandria	Status
Estratégia de testes — valor, risco e TCO	4	Não existe artefato de estratégia. As decisões de teste foram tomadas por camada técnica, não por risco de negócio	●

Padrão observável: tudo que depende de **código** está bem executado; tudo que depende de **processo e requisito** está ausente. É exatamente a assimetria que a Aula 1 descreve como "qualidade técnica alta, qualidade de processo baixa" — e o professor alerta que um processo fraco não impede um bom produto, mas reduz drasticamente a probabilidade de sustentá-lo.

3. A pirâmide de testes do Alexandria (Aula 4, §3)

Medição do que existe de fato no repositório:

Camada	O que existe	Situação
Unitários	49 arquivos de teste, ~245 <code>@Test</code> . JaCoCo: 85% instruções · 86% linhas · 84% métodos · 74% branches	✓ Base sólida — o ponto forte do projeto
Integração / API	Apenas 5 <code>*IntegrationTest</code> (Auth, Book, Job, Profile, UserBooks). Zero testes de contrato	● Camada fina
E2E / GUI	29 specs Playwright em <code>alexandria-frontend/e2e/</code>	● Existem, mas fora do Git e fora do CI
Não funcionais	k6 com 3 cenários (smoke, carga, stress) e thresholds explícitos	● Também fora do Git
Manuais / exploratórios	Não documentados	● Inexistente formalmente

O Alexandria não sofre do antipadrão "sorvete"

Vale dizer isso explicitamente na sessão. O antipadrão descrito na Aula 4 §3 — pirâmide invertida, excesso de teste manual no topo, base rala — **não é o caso aqui**. A base é robusta e a automação de GUI não é instável por excesso.

O problema é diferente e mais sutil: **a pirâmide tem o topo desacoplado do processo**. As camadas existem, mas as duas superiores não são versionadas, não são revisadas em PR, não rodam para o resto do time e não bloqueiam nada.

Evidência: as camadas superiores estão no `.gitignore`

`.gitignore:119-126` ignora explicitamente:

```
/alexandria-frontend/e2e
alexandria-frontend/playwright.config.ts
/alexandria-frontend/playwright-report
```

```
/alexandria-frontend/test-results  
/load-tests  
k6.exe
```

Confirmado: `git ls-files load-tests` retorna **0 arquivos**. Toda a camada E2E e toda a camada de teste não funcional — justamente as que a Aula 4 usa para discutir estratégia — **não estão versionadas**.

Evidência: o job de CI promete uma camada que não entrega

O `.github/workflows/ci.yaml` possui um job nomeado:

"Integração — Docker Compose + Playwright"

Esse job sobe o stack via `docker-compose.ci.yaml`, faz health check em `/actuator/health`, coleta logs em caso de falha e derruba o stack. **Ele nunca executa o Playwright**. O nome anuncia uma camada de teste que o pipeline não possui.

Isso é, no pipeline, o equivalente exato do que a Aula 3 §12–13 descreve como "*cobertura sem rastreabilidade vira volume*": um indicador que existe para ser visto, não para reduzir risco.

Gates frouxos adicionais

- `npm run lint` roda com `continue-on-error: true` — falha de lint não bloqueia nada.
- `deploy-aws.yaml` dispara automaticamente após CI verde em `develop/main` e publica em produção (bibliotecaalexandria.com.br) **sem gate de E2E e sem smoke test pós-deploy** (contraria Aula 4 §12).
- O Quality Gate do SonarCloud está vermelho há mais de uma semana e não bloqueia o deploy (ver `docs/analise-sonarcloud.md`).

4. Regra de negócio validada só no front-end (Aula 4, §4)

O professor observa que é comum, na prática, encontrar regras de validação implementadas apenas no front-end, sem nenhuma validação equivalente nos testes de API ou unitários. **O Alexandria tem um caso vivo, documentado e com defeito real:**

Aspecto	Detalhe
Regra	Cálculo de progresso e percentual de leitura do EPUB
Onde vive	Exclusivamente no frontend: <code>alexandria-frontend/src/hooks/useEpub.ts</code> , <code>src/lib/epub-cache.ts</code> , <code>src/app/(main)/leitor/[id]/page.tsx</code>
Cobertura na base da pirâmide	Nenhuma — sem teste unitário, sem teste de API
Defeito conhecido	Percentual diverge conforme o nível de zoom; barra de progresso inconsistente
Evidência	<code>docs/porcentagemxleitor/logica_zoom_paginas.md</code> + 7 screenshots comparativos (zoom 50% vs 100%)

Aspecto	Detalhe
Sinal independente	O SonarCloud aponta o único CRITICAL do projeto (<code>typescript:S2004</code> , aninhamento > 5 níveis, tag <code>brain-overload</code>) exatamente em <code>leitor/[id]/page.tsx:155</code>
Trabalho em andamento	Branch <code>feature/restaurar-posicao-leitor-epub</code> — correção de bug nessa mesma área

Este é o melhor exemplo que o grupo pode levar para a sessão: uma **regra de negócio real, sem cobertura na base da pirâmide, com complexidade excessiva sinalizada por análise estática e defeito reproduzível em produção**. Vale mais que o exemplo hipotético de transferência bancária usado em aula, porque é o próprio projeto.

5. Estratégia como artefato vivo — a mudança PDF → EPUB (Aula 4, §7)

O professor cita o caso de uma turma anterior que refez a arquitetura do sistema e precisou montar um regressivo completo, criando um épico específico para abrigar essa frente. Em contraste, times que mudaram a arquitetura de forma pontual refizeram apenas os testes das histórias afetadas.

O Alexandria passou por uma mudança de escopo equivalente e não reavaliou nada:

Artefato	O que diz	O que o sistema faz
<code>docs/documentacao.md:87</code>	RF08 — "O sistema deve oferecer um leitor de PDF integrado"	Leitor EPUB via <code>react-reader</code>
<code>README.md</code> (2 tabelas)	"Leitor de PDF"	Rota <code>/leitor/[id]</code> , proxy <code>app/api/epub/route.ts</code> , cache de EPUB
Análise de impacto regressivo	Inexistente	—

O requisito nunca foi atualizado, o impacto nunca foi analisado, nenhum épico de regressão foi criado — e a área resultante é justamente a que concentra os bugs abertos.

A pergunta-guia da Aula 4 §7 — "*a estratégia atual ainda protege o produto que está emergindo?*" — tem resposta objetiva no Alexandria: **não**.

6. Rastreabilidade e risco remanescente (Aulas 2 §6, 3 §12, 4 §10)

O `docs/relatorio-testes-parcial.md` registra a execução dos 29 testes E2E: **6 passaram, 23 falharam** por indisponibilidade do backend, e conclui que "*este comportamento é esperado*".

Do ponto de vista da disciplina, essa conclusão é o ponto mais frágil do relatório. A leitura correta é:

23 fluxos críticos ficaram sem evidência de execução.

Isso é **risco remanescente** puro (Aula 4 §10). E como não existe vínculo requisito → caso de teste, o time **não tem como afirmar se esse risco é aceitável**. É literalmente o cenário que o professor descreve: *"a equipe só sabe que não conseguiu testar algumas coisas, mas não sabe se isso representa um risco real ou irrelevante."*

Estado da cadeia de rastreabilidade **Requisito** → **Caso de Teste** → **Evidência** (Aula 3 §12):

Elo	Existe?	Observação
Requisito	 Contraditório	Três definições paralelas: as 8 user stories (informais), o Jira SCRUM-17 (RF01–RF10) e docs/documentacao.md (RF01–RF10). Os IDs colidem — RF08 é "Registrar empréstimo" no Jira e "Leitor de PDF" na documentação. Nenhuma descreve o produto real
Caso de Teste	 Parcial	245 unitários + 5 integração + 29 E2E existem, mas nenhum referencia um ID de requisito
Evidência	 Parcial	Relatórios JaCoCo, surefire, k6 e SonarCloud são gerados, mas não vinculados a requisito algum
Vínculo entre os três	 Inexistente	Sem matriz de rastreabilidade. E 0 de 229 commits citam uma chave SCRUM-xx — Jira e repositório nunca foram integrados

Nota: enquanto existirem dois "RF08" significando coisas diferentes, **nenhuma matriz de rastreabilidade pode ser construída**. Consolidar uma lista única de requisitos é pré-requisito bloqueante para todo o resto. Detalhamento em **analise-jira.md** §4.

7. Requisitos não funcionais e atributos de qualidade (Aula 4, §13)

O que está bem

- **Thresholds objetivos e testáveis**, no padrão que a Aula 3 §11 exige (nada de "rápido" ou "adequado"): erro < 5%, p(95) < 2.000 ms, login p(95) < 3.000 ms.
- **Resultados fortes:** 0% de erro em todos os cenários; 50 VUs com p(95) de 157 ms; 300 VUs com 0% de erro e 85 req/s — sem ponto de ruptura identificado.
- **Arquitetura hexagonal disciplinada** (domain / application / adapter, com ports e value objects) — isso é **testabilidade como atributo arquitetural**, e é o que explica os 85% de cobertura.
- **SonarCloud + JaCoCo configurados**, exatamente a recomendação da Aula 1 §11 (o professor pede que os alunos implementem o Sonar para "tirar uma fotografia" do código).

O desalinhamento crítico

Artefato	O que define	O que é medido
RNF01 (docs/documentacao.md)	"A interface deve carregar em menos de 3 segundos"	—
Thresholds k6	—	Latência de API (p95 < 2s)

O requisito não funcional documentado e o teste não funcional executado medem coisas diferentes.

RNF01 fala de tempo de carregamento de interface; o k6 mede tempo de resposta de endpoint. Ninguém validou RNF01. É o caso clássico de cobertura que "parece existir" mas não atende ao objetivo do requisito (Aula 3 §13).

Mapa de atributos de qualidade (Aula 4 §13) aplicado ao Alexandria

Categoria	Coberto	Descoberto
Execução	Desempenho (k6), disponibilidade (<code>/actuator/health</code>)	Resiliência, escalabilidade real (manifests K8s existem mas nunca foram testados), chaos/DR
Segurança e dados	JWT, Spring Security, RBAC planejado, 0 security hotspots	Nenhum teste de segurança ; sem threat modeling; sem OWASP; 14 vulnerabilidades no próprio pipeline
Integração	Integração Gutendex testada (<code>GutendexClientTest</code> , <code>GutendexMapperTest</code>)	Zero testes de contrato
Evolução	Manutenibilidade e testabilidade medidas (Sonar rating A + JaCoCo)	Cobertura não reportada ao Sonar; backend Java fora da análise estática
Operação e uso	Logback configurado	Sem observabilidade em produção, sem teste sintético, sem smoke pós-deploy

O professor reforça que **não é necessário maximizar todos os atributos** — é preciso priorizar conscientemente os que protegem os riscos reais. O problema do Alexandria não é ter lacunas; é que a priorização **nunca foi decidida nem documentada**. As lacunas são acaso, não escolha.

8. Tipos de teste complementares (Aula 4, §12)

Tipo	Situação no Alexandria
Regressivo	🔴 Não existe seleção por impacto. Nenhum regressivo foi montado após a troca PDF → EPUB
Exploratório	🔴 Não praticado nem registrado — apesar de o professor apontá-lo como especialmente útil nas primeiras sprints
Sanity / smoke	🟡 Existe <code>smoke.js</code> (k6), mas não roda após o deploy — justamente o momento em que a Aula 4 §12 o posiciona
Funcional	🟡 Coberto indiretamente pelos E2E (que não rodam no CI)
Não funcional	🟡 k6 bem construído, mas fora do Git e desalinhado dos RNFs

9. Análise estática — SonarCloud (Aula 1 §11 e ponte para a Aula 5)

Análise completa e detalhada em `analise-sonarcloud.md`. Resumo do que importa para a sessão:

Indicador	Situação
Quality Gate	● ERROR (falha em <code>new_security_rating</code> = C)
Bugs / Vulnerabilidades / Code Smells	3 / 16 / 52
Backend Java analisado	✗ Não — <code>ncloc_language_distribution</code> não contém <code>java</code>
Cobertura reportada	✗ Não — os 85% do JaCoCo não chegam ao Sonar
Branches analisadas	Apenas <code>main</code> , última análise em 31/07/2026

Causa raiz: três configurações conflitantes de `projectKey`, nenhuma apontando para o projeto que realmente existe (`mba-puc-alexandria_alexandria`), somadas ao uso da scan-action genérica — que não compila Java e por isso ignora os `.java`.

Leitura para a disciplina: o professor recomendou o Sonar na Aula 1 para tirar "uma fotografia do código no início e outra no fim do semestre". Hoje o Alexandria tem uma fotografia **de 3.300 linhas de TypeScript** — o backend inteiro está fora do enquadramento. E o argumento mais forte do projeto (85% de cobertura) não aparece em nenhum indicador.

Ponte para a Aula 5: o professor adiantou que as próximas aulas tratarão de complexidade ciclomática via Sonar. O Alexandria já tem o dado — complexidade ciclomática 499, cognitiva 258 — mas medido só no frontend, e com o único CRITICAL apontando para a tela do leitor EPUB, onde há bugs reais. É material pronto para essa discussão.

10. Consolidação dos riscos

#	Risco	Pilar violado	Severidade
R0	Três definições de requisito paralelas e divergentes (user stories × Jira × documentação), com IDs colidentes	Aula 2 §9 (consistência)	● Bloqueante
R1	As 8 histórias existentes nunca foram registradas em ferramenta; personas não mapeadas; nenhuma história para o leitor EPUB	Aulas 2 §9–11, 3 §9–11	● Alta
R1b	Épico de Empréstimo (US07–US08) nunca implementado, mas a tela <code>/emprestimos</code> está em produção	Aula 4 §10	● Média
R2	Ferramenta de gestão abandonada: Sprint 4 aberta há 77 dias além do prazo, sem registro do trabalho de jun–ago	Aulas 2 §5, 3 §6–7	● Alta
R3	Rastreabilidade requisito → caso de teste → evidência inexistente; 0 de 229 commits vinculados ao Jira	Aulas 2 §6, 3 §12, 4 §10	● Alta
R4	Camadas E2E e de carga fora do controle de versão	Aula 4 §3, §8	● Alta
R5	Job de CI nomeado "Playwright" não executa Playwright	Aula 3 §13	● Alta
R6	Backend Java fora da análise estática; cobertura não reportada	Aula 1 §11	● Alta

#	Risco	Pilar violado	Severidade
R7	Deploy automático em produção sem gate de E2E, Sonar ou smoke	Aula 4 §12	● Alta
R8	Mudança PDF → EPUB sem reavaliação de impacto regressivo	Aula 4 §7	● Média
R9	Regra de progresso do EPUB validada só no front, com bug aberto	Aula 4 §4	● Média
R10	RNF01 não é medido pelo que o k6 testa	Aula 3 §11, 13	● Média
R11	23 fluxos E2E sem evidência, sem análise de risco remanescente	Aula 4 §10	● Média
R12	Sem observabilidade, teste sintético ou análise de causa raiz em produção	Aulas 1 §11, 4 §9	● Média
R13	Nenhum rito de shift left (refinamento, três amigos, DoR/DoD)	Aulas 1 §10, 3 §5	● Média

11. Plano de ação

Correções de baixo custo e alto impacto

#	Ação	Efeito	Esforço
1	Remover <code>e2e/</code> , <code>playwright.config.ts</code> e <code>load-tests/</code> do <code>.gitignore</code> e versioná-los	Torna as camadas superiores da pirâmide visíveis, revisáveis e executáveis pelo time	15 min
2	Fazer o job <code>integration</code> do CI executar de fato <code>npx playwright test</code>	O pipeline passa a entregar a camada que já anuncia	30 min
3	Atualizar RF08 (PDF → EPUB) e listar as histórias impactadas pela troca	Fecha o gap de estratégia como artefato vivo	30 min
4	Corrigir a configuração do Sonar (itens 1–5 de <code>analise-sonarcloud.md</code>)	Traz backend Java + 85% de cobertura para a análise	~1h
5	Rodar <code>smoke.js</code> automaticamente após o deploy	Implementa o sanity check da Aula 4 §12	30 min

Estruturação — depende de decisão do grupo

#	Ação	Efeito	Esforço
6	Eleger as user stories como fonte única de requisito e descontinuar as duas listas RF — partindo do artefato mais bem formado que o grupo já tem	Remove o risco bloqueante R0 — pré-requisito de todo o resto	1h

#	Ação	Efeito	Esforço
7	Decidir: retomar o Jira ou migrar para GitHub Projects, e reorganizar os épicos por valor de negócio (os épicos das user stories já servem de base)	Responde à pergunta 1 do professor e viabiliza métricas de cobertura	2h
8	Diferenciar as personas nas 8 histórias usando os 4 perfis de "Público-Alvo", e desenhar 2 jornadas ponta a ponta	Responde à pergunta 3	2h
9	Escrever as ~6 histórias faltantes (leitor EPUB, progresso, dashboard, Google Auth, configurações) e converter todos os critérios para BDD/Gherkin	Responde à pergunta 2 e cobre o produto real	5h
10	Montar a matriz de rastreabilidade RF → caso de teste → evidência	Responde à pergunta 4 e viabiliza análise de risco remanescente	2h
11	Adotar a convenção de citar a chave da issue em commits e branches	Fecha o gap Jira ↔ repositório daqui para frente	5 min
12	Redigir o artefato de estratégia de testes (mesmo que uma tabela simples)	Atende ao ponto central da Aula 4 §5 — o artefato não precisa ser formal, mas precisa existir	2h

Sobre a ferramenta (item 7): a decisão não é mais "escolher do zero", e sim **retomar o Jira × migrar para GitHub Projects**. O Jira já tem 27 issues, 4 sprints e decisões registradas, além da estrutura nativa Test Plan → Test Suite → Test Case que a Aula 3 §7 usa como referência — mas foi abandonado uma vez, e há uma razão para isso. O GitHub Projects tem menos atrito (o time já vive lá) e resolve sozinho o vínculo issue ↔ PR ↔ commit, que é o gap mais grave depois do conflito de requisitos. Comparação completa em [analise-jira.md](#) §9. **Vale levar essa decisão para a sessão e ouvir a recomendação do professor.**

12. Pauta sugerida para a sessão

Atualizado após a criação de [user-stories.md](#). A sessão é uma revisão, não uma apresentação avaliativa, e o tempo é curto — a estratégia é abrir pelo resultado e sustentar a conversa com **um artefato central**, não narrar cada documento de apoio em sequência. Os demais documentos ([analise-jira.md](#), [analise-sonarcloud.md](#)) ficam prontos para abrir só se o professor perguntar.

- 1. Abrir com a síntese, em uma frase.** *"Somos fortes tecnicamente — 85% de cobertura, arquitetura hexagonal, k6 com 0% de erro a 300 VUs — e fracos em processo: testamos bem, mas não sabíamos dizer o que estávamos testando."* Define o enquadramento em segundos e evita parecer que o grupo está escondendo algo.
- 2. Mostrar um artefato só: [user-stories.md](#).** É o que mais rende no pouco tempo — prova ao mesmo tempo que o grupo sabe escrever história de usuário (formato correto, INVEST, épicos por domínio de negócio, não por camada técnica) e mostra exatamente onde o processo quebrou (as histórias nunca chegaram ao Jira). Sozinho, já responde 3 das 4 perguntas do item 15 da Aula 4.

3. **Aprofundar em um único achado: as três fontes de requisito conflitantes.** Jira ([SCRUM-17](#)), [docs/documentacao.md](#) e as próprias [user-stories.md](#) — cada uma com um RF08 diferente (Registrar empréstimo / Leitor de PDF / não existe RF08 nestas histórias). É concreto, rápido de explicar, e é a causa raiz de quase tudo o resto (rastreabilidade, cobertura, risco remanescente). Não é preciso desenrolar os demais achados (tela fantasma de empréstimo, Sonar, CI que anuncia Playwright sem rodar) a menos que o professor pergunte — mantê-los como "temos isso documentado" é suficiente.
4. **Fechar pedindo orientação em duas decisões, não com conclusões prontas** — transforma a sessão em conversa, não confissão:
 - qual das três fontes vira a lista única de requisitos daqui para frente;
 - retomar o Jira ou migrar para GitHub Projects.
5. **Ter os documentos de apoio prontos, mas não narrados.** Se o professor quiser ir mais fundo em algum ponto específico — priorização de atributos de qualidade (Aula 4 §13), o caso do leitor EPUB, granularidade de Features vs. histórias — abrir na hora a partir de [analise-jira.md](#), [analise-sonarcloud.md](#) ou das seções 3–7 deste documento.

13. Síntese

Dimensão	Avaliação
Qualidade técnica (Aula 1 §5)	● Forte — hexagonal, 85% de cobertura, k6 robusto, 0,9% de duplicação, Sonar rating A de manutenibilidade
Qualidade de produto (Aula 1 §5)	● Média — o produto funciona e está em produção, mas as três definições de requisito que o descrevem (user stories, Jira, documentação) são contraditórias entre si e desatualizadas
Qualidade de processo (Aula 1 §5)	● Fraca — ferramenta de gestão abandonada, sem rastreabilidade, sem estratégia de testes, sem ritos de shift left

A frase que resume o projeto para a sessão de sábado:

O Alexandria testa bem, mas não sabe dizer o que está testando — e é exatamente isso que a disciplina se propõe a corrigir.

Referências

- [resumo-aula1-testes-de-software.md](#) — Qualidade, Shift Left e o Processo de Produto
- [resumo-aula2-testes-de-software.md](#) — Requisitos, Equipes, Estrutura de Projeto e Qualidade de Histórias
- [resumo-aula3-testes-de-software.md](#) — Planejamento e Testabilidade
- [resumo-aula4-testes-de-software.md](#) — Estratégia de Testes, Pirâmide e Tipos de Teste
- [user-stories.md](#) — As 8 histórias de usuário, avaliação INVEST e status de implementação
- [analise-jira.md](#) — Análise detalhada do backlog no Jira
- [analise-sonarcloud.md](#) — Análise detalhada do SonarCloud

- [relatorio-testes-parcial.md](#) — Relatório parcial de testes (E2E, carga, cobertura)
 - [documentacao.md](#) — Documentação funcional (RF01–RF10, RNF01–RNF07)
 - [porcentagemxleitor/logica_zoom_paginas.md](#) — Análise dos bugs de percentual do leitor EPUB
-

Documento elaborado em 08/08/2026 a partir dos resumos das Aulas 1 a 4 da disciplina de Testes de Software e da análise do repositório Alexandria.